Cloud-Based Resume Analyzer & ATS Scoring System

Comprehensive Technical Architecture, Database Specifications, and Faculty Viva Defense Handbook

Project Name:	Cloud-Based Resume Analyzer with ATS Scoring	**Backend Stack:**	Java 21, Spring Boot 3.3.4, JPA/Hibernate
Primary Author:	Kothapalli Bhaskar	**Database:**	MySQL 8.0 (Production) / H2 (In-Memory Fallback)
Frontend Stack:	React 18 SPA, Vite, Axios, Lucide Icons	**Document Engine:**	Apache PDFBox 3.0.3, Apache POI 5.3.0 (Zero LLMs)

1. Executive Summary & The 30-Second Elevator Pitch

The Elevator Pitch:
*'Sir/Madam, this project is an enterprise-grade, cloud-scalable Resume Analyzer and Applicant Tracking System (ATS) scoring platform built on Java Spring Boot 3 and React 18. Unlike ordinary student projects that merely invoke external generative AI APIs (which suffer from hallucinations, non-deterministic scores, high API costs, and severe privacy violations when handling personal resume data), our system implements **100% deterministic mathematical set algorithms**, **standard document parsing libraries (Apache PDFBox and Apache POI)**, and **RFC-compliant regular expressions**. It evaluates candidate resumes against job descriptions with complete transparency, auditability, and speed.'*

2. System Architecture & Component Interactions

The project follows an enterprise **3-Tier Layered Architecture** ensuring strict separation of concerns, stateless horizontal scalability, and cloud resilience:

Tier / Layer	Components & Technologies	Responsibilities
Presentation Tier (Client Frontend)	React 18 SPA, Vite, Axios, Lucide Icons, Glassmorphic CSS	Multi-step guided interface: (1) Target Role & JD entry, (2) Resume Upload with instant preview, (3) Score gauges & diagnostics breakdown, (4) ATS-optimized resume exporter.
Application Tier (Backend REST API)	Java 21, Spring Boot 3.3.4, Spring Security, JWT (io.jsonwebtoken), Jackson	Stateless authentication, multipart file upload routing, business logic orchestration, RESTful API endpoints (/api/resumes, /api/auth, /api/health).
Processing Engines (Deterministic Core)	Apache PDFBox 3.0.3, Apache POI 5.3.0, java.util.regex.Pattern	Raw text extraction from PDF & DOCX; contact information mining (Email, Phone, LinkedIn, GitHub); 7-section boundary header detection; ATS mathematical scoring.

Data & Object Storage Tier (Persistence Layer)	MySQL 8.0 / MariaDB (Hibernate JPA) AWS S3 (AWS SDK v2) / Local File System	Relational persistence of users, resume metadata, and ATS analysis reports. Resume binary documents stored securely in AWS S3 or fallback local disk.

3. Database Architecture & Schema Specification

When your faculty asks: *'Where is the database and how is it structured?'*, this section provides your exact technical answer.

Configuration File: backend/src/main/resources/application.properties
The project connects to MySQL via Spring Data JPA and Hibernate ORM. It also features an embedded H2 fallback for zero-dependency standalone demonstrations.

Table Name	Key Columns	Constraints & Relationships	Description
users	id (PK, Auto) username email password created_at	UNIQUE(username) UNIQUE(email) BCrypt encrypted password	Stores candidate/recruiter accounts.
user_roles	user_id (FK) role	PK(user_id, role) FK -> users(id) ON DELETE CASCADE	Role-based access control (ROLE_USER, ROLE_ADMIN).
resumes	id (PK, Auto) file_name file_format file_size storage_type storage_path_or_key raw_extracted_text user_id (FK) uploaded_at	FK -> users(id) ON DELETE SET NULL storage_type: 'S3' | 'LOCAL' raw_extracted_text: LONGTEXT	Stores resume file metadata and full extracted text. **Notice:** Binary files are NOT kept in SQL to prevent bloat; paths/keys are referenced instead.
analysis_reports	id (PK, Auto) resume_id (FK) job_title job_description overall_score skill_match_score section_score keyword_frequency_score formatting_score matched_skills missing_skills contact_info_json analyzed_at	FK -> resumes(id) ON DELETE CASCADE Scores: 0 to 100 integer bounds JSON/Text serialized arrays	Stores historical ATS scoring audits, diagnostic sub-scores, detected skills, and recommendations.

4. The Deterministic ATS Engine & Mathematical Formulas

The application relies on **deterministic mathematical algorithms** rather than non-reproducible generative models. This mirrors real-world enterprise ATS behavior (Workday, Taleo, Greenhouse).

The Weighted Composite Score Formula (0 - 100):

Composite ATS Score = (0.50 × Skill_Match) + (0.20 × Section_Score) + (0.15 × Keyword_Density) + (0.15 × Formatting_Score)

1. Skill Match Score (50% Weight) — Mathematical Set Theory:

Let S_{resume} be the set of skills extracted from the candidate's resume, and S_{job} be the set of skills demanded by the target job description.

- **Matched Skills (Set Intersection):** $S_{matched} = S_{resume} \cap S_{job}$
- **Missing Skills (Set Difference):** $S_{missing} = S_{job} \setminus S_{resume}$
- **Calculation:** $Skill_Match_Score = (|S_{matched}| / |S_{job}|) \times 100$

2. Section Completeness Score (20% Weight):

The system evaluates the resume against 7 standard industry sections: *Contact Information, Summary, Work Experience, Education, Skills, Projects,* and *Certifications* using high-precision regex header patterns. Score = (Detected_Sections / 7) × 100.

3. Keyword Frequency & Anti-Stuffing Density Score (15% Weight):

Density is computed as: $Keyword_Density = (Total_Skill_Occurrences / Total_Words) \times 100$

- **Optimal ATS Density (1.5% to 6.0%):** Score = 100 points.
- **Mild Keyword Stuffing (6.0% to 9.0%):** Penalized to 75 points.
- **Aggressive Keyword Stuffing (> 9.0%):** Heavily penalized to 55 points.

This defends recruiters against fraudulent candidates who hide white-text repetitive keywords!

4. Formatting & Readability Score (15% Weight):

Starts at 100. Evaluates optimal length (400 - 1000 words). Penalties are levied if resume is <150 words (-40) or >1500 words (-25). Mandatory contact audits deduct points if email (-20), phone (-15), or LinkedIn/GitHub (-10) are missing.

5. Strict Faculty Defense: Rapid-Fire Viva Q&A;

Prepare these exact answers when your faculty probes you with difficult questions:

Q1: Where is your database? Open it and show me the tables right now!

Answer: 'Sir, our database is fully configured in application.properties. For production, we connect to MySQL on localhost:3306 under database resume_analyzer_db. For local testing, we have an active H2 web console at http://localhost:8080/h2-console. There are 4 relational tables: users, user_roles, resumes, and analysis_reports with proper primary and foreign keys.'

Q2: Why didn't you use OpenAI, Gemini, or an LLM API to parse the resume?

Answer: 'Sir, that was an intentional design decision for 3 critical enterprise reasons: (1) Data Privacy & GDPR: Resumes contain sensitive Personally Identifiable Information (phone, address, email); transmitting them to commercial LLM APIs introduces data leakage risks. (2) Determinism: LLMs are non-deterministic and hallucinate different scores across runs; our set-theory math produces 100% reproducible audits. (3) Latency & Cost: Native Java parsers execute in under 120 milliseconds with zero API costs.'

Q3: How do you extract text from PDF and DOCX files without OCR or AI?

Answer: 'Sir, we utilize Apache PDFBox 3.0.3 using Loader.loadPDF() and PDFTextStripper for PDF binary streams, and Apache POI 5.3.0 with XWPFDocument and XWPFWordExtractor for DOCX files. Control characters are sanitized through standard Java regex streams.'

Q4: How did you implement security and user authentication?

Answer: 'Sir, we implemented Spring Security with stateless JSON Web Tokens (JWT). Passwords are never stored in plain text—they are hashed with BCrypt. Protected endpoints require an Authorization: Bearer header, validated on every request by our JwtAuthenticationFilter without server session state.'

Q5: What happens if someone uploads a 50MB file or an encrypted PDF?

Answer: 'Sir, we handle this defensively: (1) Spring Boot enforces a 15MB multipart ceiling via spring.servlet.multipart.max-file-size=15MB. (2) In DocumentParserService, we invoke document.isEncrypted() on Apache PDFBox; if encrypted, an IOException is immediately thrown and returned as a HTTP 400 Bad Request.'

Q6: How does the system prevent candidates from cheating via Keyword Stuffing?

Answer: 'Sir, in AtsScoringEngineService, we compute Keyword Density: (matched keyword occurrences / total word count) * 100. If a candidate repeats keywords to exceed 6%, our engine penalizes their score from 100 down to 75, and if over 9%, down to 55.'

6. Teammate Guide: Database Setup & Teamwork Division

Give this exact instruction set to your friend to create and manage the project's MySQL database:

Step 1: Starting MySQL via XAMPP

1. Open the **XAMPP Control Panel** on your system.
2. Click **Start** next to **MySQL** and **Apache** (Ports 3306 & 80).
3. Open your browser and navigate to: http://localhost/phpmyadmin

Step 2: Database Initialization Script (schema.sql)

```
CREATE DATABASE IF NOT EXISTS resume_analyzer_db;
USE resume_analyzer_db;

CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_roles (
  user_id BIGINT NOT NULL,
  role VARCHAR(50) NOT NULL,
  PRIMARY KEY (user_id, role),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE resumes (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  file_name VARCHAR(255) NOT NULL,
  file_format VARCHAR(10) NOT NULL,
  file_size BIGINT,
  storage_type VARCHAR(20) NOT NULL,
  storage_path_or_key VARCHAR(500) NOT NULL,
  raw_extracted_text LONGTEXT,
  user_id BIGINT,
  uploaded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE analysis_reports (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  resume_id BIGINT NOT NULL,
  job_title VARCHAR(255),
  job_description LONGTEXT,
  overall_score INT NOT NULL,
  skill_match_score INT NOT NULL,
  section_score INT NOT NULL,
  keyword_frequency_score INT NOT NULL,
  formatting_score INT NOT NULL,
  word_count INT NOT NULL,
  matched_skills TEXT,
  missing_skills TEXT,
  present_sections TEXT,
  missing_sections TEXT,
  contact_info_json TEXT,
  recommendations_json TEXT,
  analyzed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (resume_id) REFERENCES resumes(id) ON DELETE CASCADE
);
```

Step 3: Team Division of Responsibilities (For Faculty Review)

Team Member	Assigned Responsibilities	What to Say in Viva
Kothapalli Bhaskar (You)	Backend services, document extraction pipeline (Apache PDFBox/POI), deterministic ATS scoring engine mathematics, and React SPA dashboard.	*'I engineered the document parser integrations, implemented the mathematical scoring algorithms, and built the interactive React dashboard.'*
Your Teammate (Friend)	Database schema architecture, normalization (3NF), relational table constraints, SQL indexing, and database migration testing.	*'I architected the relational database schema, established foreign-key cascades, and optimized storage performance for unstructured resume text.'*

Quick Execution Commands for Faculty Demo:

- **Start Backend:** cd 'd:\cloud project\backend' ; .\mvnw.cmd spring-boot:run
- **Start Frontend:** cd 'd:\cloud project\frontend' ; npm run dev
- **Inspect Database:** http://localhost/phpmyadmin (or http://localhost:8080/h2-console)
- **Live Application:** http://localhost:5173